

MAS Investor Alert List Compliance Agent

System Architecture & Implementation Guide

Document Date: 29 July 2026

HM Strategy

Two-system IAL compliance screening: Notification Engine + Streamlit Screening UI

The HM Strategy MAS IAL Compliance Agent is a two-component system designed to screen entities against the Monetary Authority of Singapore's (MAS) Investor Alert List (IAL). The system processes 622+ unregulated entity records spanning 2017 to present, sourced directly from MAS's internal search API.

Two-Component Architecture

- Notification Engine (main.py): Automated cron-driven system that scrapes the MAS IAL every 48 hours, detects new entries, and emails Fred with a compliance report.
- Screening UI (app.py): Interactive Streamlit web application for on-demand name screening with bulk upload and individual search capabilities.

Key Design Principles

- Silence rule: No email is sent when zero new entries are detected
- Zero data persistence: Client lists are never stored - processed in-memory only
- Snapshot diffing: New entry detection via persisted JSON snapshot, not API state comparison
- Graceful degradation: Falls back to local data file if MAS API is unreachable

The system sources data from the MAS Investor Alert List's internal Solr-based search API, discovered by reverse-engineering the MAS website's JavaScript bundle (bundle.search-ial.1.0.0.js). This API powers the search functionality on the public MAS IAL webpage.

API Endpoint

```
GET https://www.mas.gov.sg/api/v1/ialsearch
Parameters:
  json.nl=map      - Response format (map)
  wt=json          - Output format (JSON)
  sort=date_dt desc - Sort by date descending
  rows=1000        - Max records per request
  start=0          - Pagination offset
  q=date_dt:[2017-01-01T00:00:00Z TO *] - Date filter
```

Response Fields & Schema Mapping

API Field	Type	Maps To	Description
unregulatedpersons_s	String	name	Entity/individual name
alternativenam_s	String	aliases	Alternative names (comma-separated)
formername_s	String	aliases	Former names (comma-separated)
date_s	Date	date_added	Date added to IAL
notes_s	String	description	Free-text notes about the entry
website_s	String	website	Associated website URL
email_s	String	email	Associated email address
agency_custom_categories	String	type	Always 'Investor Alert List'

Total records: 913 (622 filtered to 2017-present). API returns all matching records in a single paginated call with rows=1000.

The codebase consists of six Python modules, each with a distinct responsibility. Modules are independently testable and can be imported separately.

Module Dependency Graph

```
app.py (Streamlit UI)
|-- scraper.py      (MAS API client)
|-- matcher.py     (Name matching engine)
|-- report.py      (PDF report generator)
main.py (Notification Engine)
|-- scraper.py      (MAS API client)
|-- matcher.py     (Name matching engine)
|-- report.py      (PDF report generator)
|-- emailer.py     (SendGrid email client)
```

3.1 scraper.py - API Client

Handles all communication with the MAS IAL Solr API. Uses the standard requests library (not Playwright or Selenium) to call the JSON endpoint directly. Key functions:

- `_do_scrape()` -> (list[dict], str): Performs the API request, parses the JSON response, maps fields to the internal schema. Returns data and source tag ('live' or 'fallback').
- `scrape_mas_ial()` -> list[dict]: Public convenience wrapper that discards the source tag.
- `_load_fallback()` -> list[dict]: Loads emergency fallback data from `data/ial_fallback.json` if the API is unreachable.
- `_normalize_date(raw: str)` -> str: Parses multiple date formats into YYYY-MM-DD.
- `_split_aliases(raw: str)` -> list[str]: Splits comma/semicolon-separated alias strings.

Fallback chain: API call -> fallback JSON file. No browser automation needed.

3.2 matcher.py - Name Matching Engine

Implements four matching strategies evaluated in order of priority. Returns the first match found.

Strategy	Confidence	Condition	Example
Exact	100%	Normalized names are identical	"Binance" == "Binance"
Alias	100%	Input matches any alias of the entry	"3AC" in Three Arrows Capital aliases
Substring	90%	One name contains the other (both >=5 chars)', "	
Fuzzy	85-99%	rapidfuzz token_sort_ratio >= threshold	Typo tolerance

Normalization pipeline: Uppercase -> strip legal suffixes (PTE LTD, LTD, INC, etc.) -> remove parentheses -> collapse whitespace. The `normalize_name()` function preserves all punctuation except brackets. Threshold default: 85 (configurable via argument).

3.3 report.py - PDF Generator

Uses the `fpdf2` library to generate branded PDF reports. Three report types:

- `generate_notification_report(new_entries, client_matches)`: For the email pipeline - lists new IAL entries and any flagged client matches from the snapshot.
- `generate_screening_report(matches, total_screened)`: For bulk upload results - summary table with color-coded confidence rows (green=100%, amber=90%, orange=<90).
- `generate_single_search_report(result)`: For individual search - shows FLAGGED or NOT FLAGGED status with full entry details.

All reports share an `MASReport` base class with brand styling: navy header, blue accent lines, page numbers, compliance disclaimer footer.

3.4 emailer.py - Email Client

Minimal SendGrid wrapper. Sends a single email with the notification PDF as attachment. Uses environment variables for credentials (SendGrid API key, recipient, sender). Has a dry-run mode for testing.

3.5 main.py - Notification Engine

The orchestration script for the automated notification pipeline. Execution flow:

```
1. Load client list from data/client_list.csv (if exists)
2. Scrape current MAS IAL data via scraper.scrape_mas_ial()
3. Load previous snapshot from data/mas_snapshot.json
4. Diff current data against snapshot to find new entries
5. Save current data as new snapshot
6. If no new entries and no --force-email flag: EXIT (silence rule)
7. If new entries found: run client names against new entries via matcher
8. Generate notification PDF via report.generate_notification_report()
9. Send email with PDF attachment via emailer.py
```

- Flags: --force-email sends an email even when zero new entries (for testing)
- Snapshot file (mas_snapshot.json) is tracked in git for GitHub Actions persistence

3.6 app.py - Streamlit Screening UI

Interactive web application deployed via Streamlit. Two-tab interface:

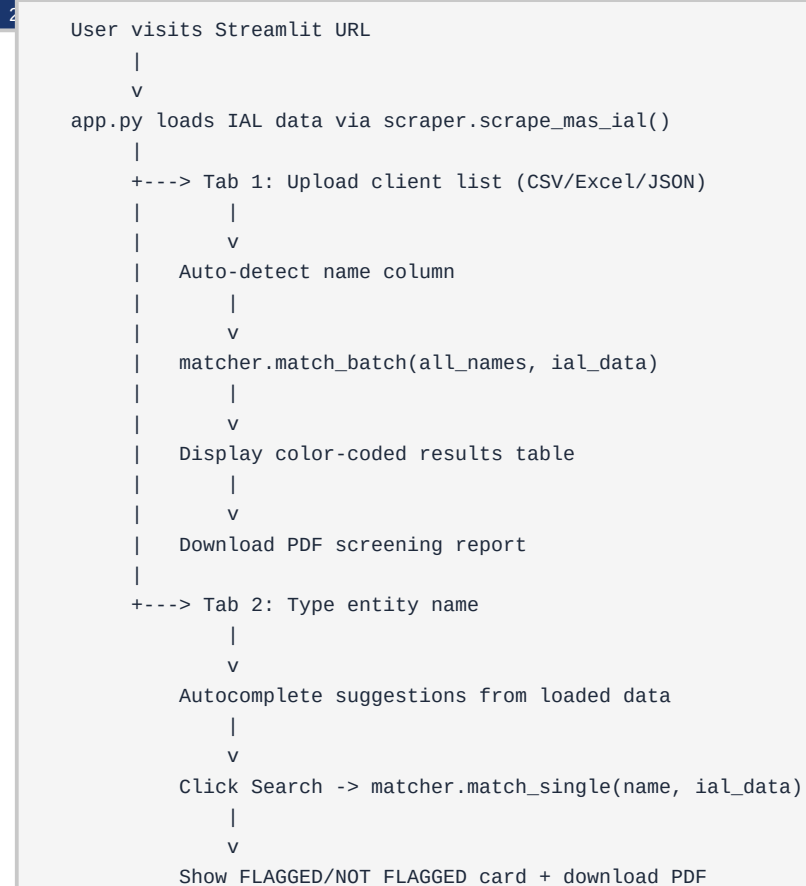
- Tab 1 - Bulk Client Screening: Upload CSV/Excel/JSON, auto-detect name column, assessor column optional. Preview data, run screening against 622+ IAL entries, download PDF report.
- Tab 2 - Individual Name Search: Type entity name (min 2 chars), select from autocomplete suggestions, view match results with confidence badge + full details (aliases, description, dates), download single-search PDF report.

Session state management: Results persist across reruns via st.session_state. File upload is suppressed during search flows via _skip_bulk flag. Suggestions computed client-side from loaded IAL data (no additional API calls).

4.1 Notification Flow (48-hour cycle)

```
GitHub Actions (cron: 0 22 */2 * *)
|
v
python main.py
|
+---> scraper._do_scrape() -----> MAS API (REST JSON)
|                                     |
|                                     Returns 622+ entries
|                                     |
+---> Load mas_snapshot.json
|     |
|     v
+---> Diff: new entries = current - snapshot
|     |
|     +-- [0 new] --> EXIT (silence rule)
|     |
|     +-- [N new] --> Continue
|                     |
|                     v
|             Match client list against new entries
|                     |
|                     v
|             Generate PDF notification report
|                     |
|                     v
|             Send via SendGrid --> Fred's email
|
+---> Write updated snapshot --> git commit & push
```

4.2 Screening Flow (User-initiated)



The matching algorithm evaluates four strategies in strict priority order. Each strategy must meet its confidence threshold; if it does, the result is returned immediately without evaluating lower-priority strategies.

5.1 Strategy 1: Exact Match (100%)

Both names are normalized (uppercased, legal suffixes stripped, parentheses removed, whitespace collapsed) and compared for string equality. This catches direct matches where the input name exactly matches an IAL entry name.

5.2 Strategy 2: Alias Match (100%)

The normalized input name is compared against each normalized alias of every IAL entry. If any alias matches exactly, it returns a 100% confidence alias match. This handles widely-known trading names and former entity names.

5.3 Strategy 3: Substring Match (90%)

Both normalized names must be at least 5 characters long. If one normalized name contains the other as a substring, it returns a 90% confidence match. This catches partial name matches like 'Binance' matching 'Binance Holdings Limited'.

5.4 Strategy 4: Fuzzy Match (85%+ threshold)

Uses rapidfuzz.fuzz.token_sort_ratio to compute a similarity score between the token-sorted versions of both normalized names. If score >= 85, the match is returned with the raw score as confidence. This is the catch-all strategy that handles typos, word ordering differences, and minor spelling variations.

5.5 Normalization Pipeline

```
Input: 'Binance Holdings Pte Ltd'
Step 1: Uppercase -> 'BINANCE HOLDINGS PTE LTD'
Step 2: Strip legal suffixes -> 'BINANCE HOLDINGS '
    Matched suffixes: PTE LTD, LTD, LIMITED, LLC, INC, CORP,
    CORPORATION, GMBH, AG, SA, PLC, SDN BHD, and 20+ more
Step 3: Remove brackets () {} [] -> 'BINANCE HOLDINGS '
Step 4: Collapse whitespace -> 'BINANCE HOLDINGS'
Output: 'BINANCE HOLDINGS'
```

6.1 GitHub Actions - Notification Engine

The notification pipeline runs on GitHub Actions with the following configuration:

- Trigger: Scheduled cron (0 22 */2 * *), equivalent to every 2 days at 6:00 AM Singapore time.
- Manual trigger: workflow_dispatch enables on-demand runs via GitHub UI.
- Environment: ubuntu-latest with Python 3.12. Dependencies installed via pip install -r requirements.txt.
- Secrets: SENDGRID_API_KEY, FRED_EMAIL, FROM_EMAIL configured as GitHub Actions repository secrets.
- Snapshot persistence: mas_snapshot.json is updated and committed back to the repository after each run.
- Git config: Masquerades as 'MAS IAL Bot' <ial-bot@hm-strategy.com> for the commit.

6.2 Streamlit Community Cloud - Screening UI

The interactive screening application is deployed on Streamlit Community Cloud:

- Free tier: Public GitHub repository, no credit card required.
- Deployment: One-click from GitHub repo via share.streamlit.io.
- Cold start: ~5 second wakeup after idle period (app auto-sleeps).
- Scaling: Handles multiple concurrent users; each session is isolated.
- Branding: Small 'Built with Streamlit' badge in footer only.
- URL format: https://<app-name>.streamlit.app (customizable).

6.3 Security Considerations

- No client data stored: Uploaded CSV/Excel/JSON is processed entirely in-memory, never written to disk or database.
- No API key exposure: SendGrid credentials are environment variables, never hard-coded.
- Public API consumption: The MAS IAL API is publicly accessible without authentication.
- Git history: .env, client list files, and __pycache__ are gitignored.

File	Lines	Purpose
app.py	~430	Streamlit screening UI (bulk + individual)
scraper.py	~140	MAS API client with fallback
matcher.py	~86	4-strategy name matching engine
report.py	~255	FPDF report generator (3 templates)
emailer.py	~60	SendGrid email wrapper
main.py	~120	Notification engine orchestrator
data/ial_fallback.json	40	Emergency fallback dataset
data/mas_snapshot.json	tracked	Snapshot for diff detection
.env.example	~5	Environment variable template
.streamlit/config.toml	~8	Streamlit server config
requirements.txt	~8	Python dependencies
.github/workflows/notify.yml	~35	GitHub Actions cron workflow

```
# .env file configuration

SENDGRID_API_KEY=SG.xxxxxx      # SendGrid API key for email
FRED_EMAIL=fred@hmstrategy.com   # Notification recipient
FROM_EMAIL=ial-bot@hmstrategy.com # Verified sender email

# GitHub Actions secrets (same keys):
#   Settings -> Secrets and variables -> Actions
#   Add: SENDGRID_API_KEY, FRED_EMAIL, FROM_EMAIL
```

```
name: MAS IAL Notification
on:
  schedule:
    - cron: '0 22 */2 * *'      # Every 2 days at 6AM SGT
  workflow_dispatch:           # Manual trigger button

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r requirements.txt
      - run: python main.py
    env:
      SENDGRID_API_KEY: ${ secrets.SENDGRID_API_KEY }
      FRED_EMAIL: ${ secrets.FRED_EMAIL }
      FROM_EMAIL: ${ secrets.FROM_EMAIL }
    - run: |
        git config user.name 'MAS IAL Bot'
        git config user.email 'ial-bot@hm-strategy.com'
        git add data/mas_snapshot.json
        git diff --cached --quiet ||
          git commit -m 'Update MAS IAL snapshot'
        git push
```

Each entry in the system conforms to this schema:

Field	Type	Example	Source
name	str	Binance	unregulatedpersons_s
aliases	list[str]	["Binance.com"]	alternativenamename_s + formername_s
type	str	Investor Alert List	agency_custom_categories
date_added	str	2021-09-01	date_s (normalized)
description	str	Entity impersonates...	notes_s
website	str	https://binance.com	website_s
email	str		email_s

The fallback JSON file (ial_fallback.json) uses the same schema and is only loaded when the primary API endpoint is unreachable. It contains 38 high-profile cryptocurrency entities as a minimal emergency dataset.

End of Document. Generated automatically by HM Strategy architecture documentation script. This document is intended for internal use and as a reference for LLM-assisted development of similar compliance screening systems.